Initiative Vision: Technical Report

Oct 22, 2026 · @Osfield

Initiative Vision is a live, cloud-processed driver-assistance pipeline. A Raspberry Pi camera in a real car streams frames over cellular to AWS, where OpenCV 5 with COOL on Graviton reads lanes, vehicles and road signs a little over a second after the camera captures each frame.

Entry for the OpenCV AI Competition 2026. Code: github.com/osfieldgaga/initiative-vision-aws (Pi side: github.com/osfieldgaga/initiative-vision). Live demo: www.initiativelabs.cc/ivision-demo (password in the submission). Judge data: release v1.0. Measured from the real car, a frame comes back processed about 1.2 to 1.4 s after the camera captured it.

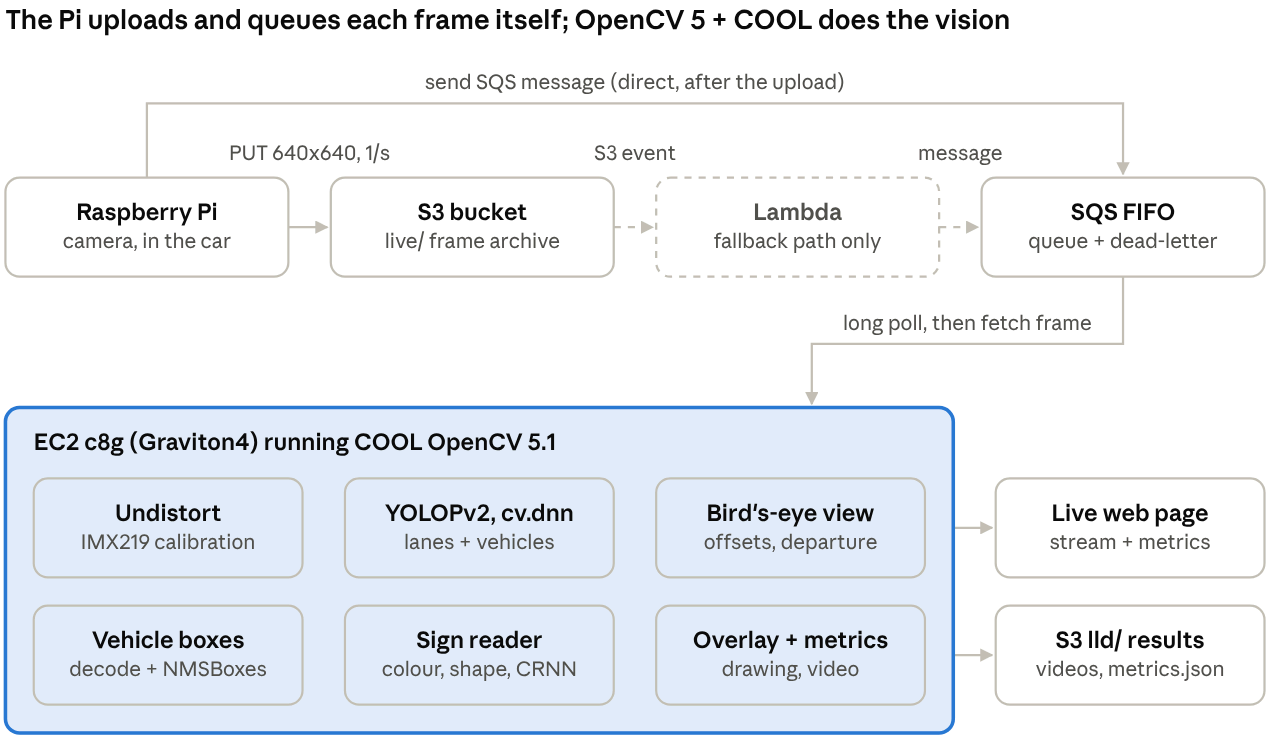
Problem and users

Most cars on the road have no lane-departure or sign-reading assistance, and retrofit units are closed boxes whose data never leaves the car. Initiative Vision moves the perception to the cloud: a cheap camera and a Pi in the car, and the heavy vision on a shared Graviton instance.

- **Drivers of older cars:** lane position, departure and sign readings without replacing the car
- **Fleets:** one cloud pipeline watching many vehicles, with every frame and result archived
- **Driving-data collection:** labelled lane, vehicle and sign data produced live, as it is driven

Beyond these use cases, the Vision 1 is intended as a data collection tool for research purposes, enabling an easier gateway into providing intelligence to vehicles. .....

Architecture



architecture · Pi to AWS to OpenCV 5 + COOL

The Pi only captures, downscales to 640 x 640 and uploads; all vision runs inside one process on the Graviton instance, which fetches each frame from S3 when its SQS message arrives. Since 2026-10-22 the Pi sends that message itself (direct mode); the S3 event to Lambda path, which added about 1 s of event delivery, remains as a fallback.

OpenCV 5 implementation

Every stage after the frame lands in S3 runs on OpenCV 5; PyTorch is no longer needed at inference time.

1. **Undistort and rotate** (`cv.undistort` with the IMX219 calibration, `getOptimalNewCameraMatrix` cached per frame size)
2. **YOLOPv2 on `cv.dnn`**: the TorchScript model exported to ONNX (`src/scripts/export_onnx.py`). The exporter packs the detection maps into an ONNX Sequence, which `cv.dnn` cannot read, so they are re-exposed as plain outputs. One forward pass gives the drivable area, lane lines and three raw detection maps.

3. **Bird's-eye view**: `warpPerspective` with a homography calibrated on a 6 x 6 m cone square (100 px per metre; y = 700 is 4.874 m ahead of the car). Lane contours in the BEV give per-side offsets, road width and departure.
4. **Vehicle detection**: the YOLOv7-style head is decoded in NumPy (sigmoid, grid and anchors) and de-duplicated with `cv.dnn.NMSBoxes`.
5. **Sign reader** (`signs.py`), in three stages:
   a. Colour: an HSV red mask for STOP; Canny edges for white speed-limit signs, because a white sign on a bright sky has no colour edge
   b. Shape: an octagon-like red blob (6 to 10 hull vertices, solid, about square), or an upright 4:5 quadrilateral with a bright face and dark legend
   c. OCR: OpenCV's CRNN text recogniser (model zoo, on `cv.dnn`). Instead of trusting the free-form reading, the CTC forward algorithm scores the probability of each legal text ("stop", or 15 to 85 mph in steps of 5) against the best free-form path. A blurry 60 px sign read as "stoe" still scores as STOP; a random blob does not.
6. **Overlay and outputs**: drawing, video writing and JPEG encoding for the live page, all OpenCV.

The ONNX model matches TorchScript exactly on 40 real frames: lane and drivable-area mask IoU 1.0000, the same detections, and identical metrics (`src/scripts/compare_onnx.py`).

AWS deployment and COOL

The vision workload runs on COOL's OpenCV 5.1.0-dev on a Graviton4 c8g.xlarge, and switching its DNN path to the classic engine with an Arm FP16 target cut YOLOPv2 inference from 1.20 s to 0.74 s per frame (measured 2026-10-22).

Deployment (us-east-1, all event-driven):

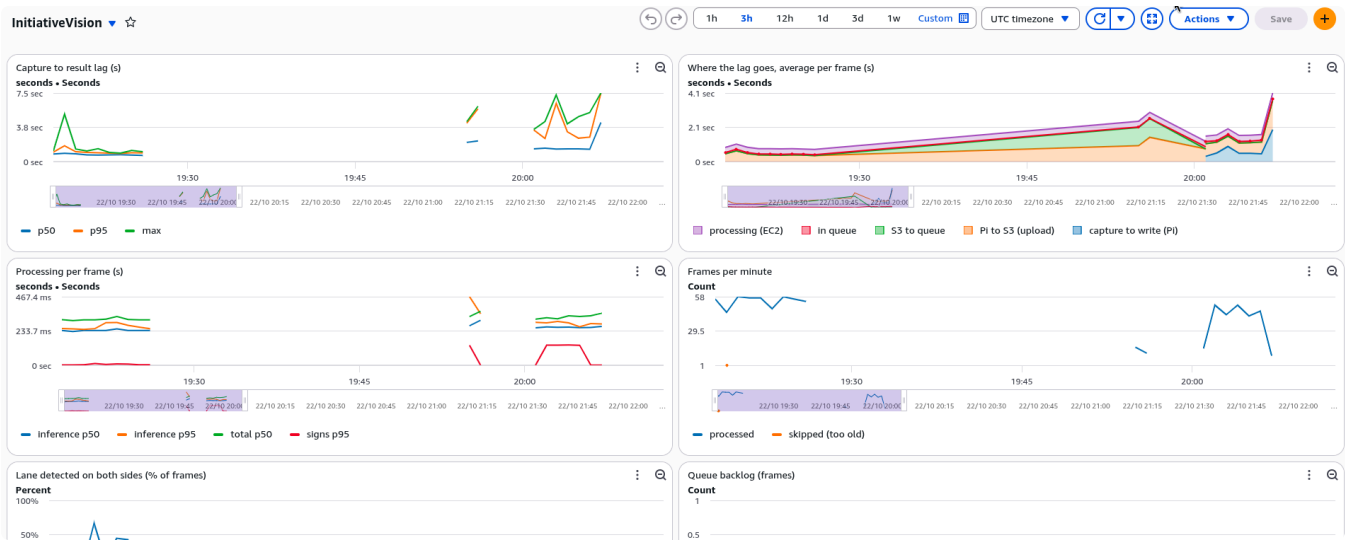
Live path (direct mode): the Pi downscales each frame to 640 x 640 JPEG q70 (about 25 KB, 73 ms on the Pi 3B), `PUT` s it to its archive key `live/<session>/<frame time>.jpg` , then sends the SQS message itself in the same format the Lambda used. `lld.py` scales the camera calibration to the frame size. The bullets below are the original Lambda path, kept as the fallback.

- The Pi `PUT` s each frame to S3 bucket `initiative-vision-1` as `live/latest_frame.jpg` (1280 x 1280, JPEG q70, at most 1 upload/s), with session, frame time and sequence number as object metadata

- An S3 event triggers Lambda `on-live-frame`, which copies that exact object version to `live/<session>/<frame time>.jpg` and sends a message to SQS FIFO `lld-frames.fifo` (dead-letter queue after 5 receives, 120 s visibility)
- EC2 runs `lld_aws.py live` in COOL's venv: it long-polls SQS, processes each frame and serves the live page; when a session ends it writes `lld/<session>/overlay.mp4`, `bev.mp4` and `metrics.json`
- When the queue backs up, live mode skips frames older than `LLD_MAX_LAG_S` (the archive keeps them all)
- IAM roles are least-privilege per component; S3 versioning is on

Operations:

- **Observability:** every 10 s, `lld_aws.py` sends per-frame values to the CloudWatch namespace `InitiativeVision`: end-to-end lag and each hop (capture to write, Pi to S3, S3 to queue, in queue), inference, sign reading and processing time, frames processed and skipped, lane-valid % and queue backlog. Timings go as value lists, so the dashboard (`infra/cloudwatch-dashboard.json`) shows p50 and p95, not just averages. The instance role may only `PutMetricData` into that namespace. A failure to publish never stops processing.
- **Outputs that play in a browser:** session videos are converted to H.264 with ffmpeg when a session closes (OpenCV's pip wheels can't write H.264).
- **Reproducibility:** pinned `requirements.txt` (EC2, inside COOL's venv) and `requirements-dev.txt` (laptop), a step-by-step setup guide, and 9 test files run with `unittest`.



The CloudWatch dashboard during the 2026-10-22 runs: capture-to-result lag (p50/p95/max), where the lag goes per hop, processing per frame, and frames per minute. The late spike to about 7 s at p95 came from the Pi side (Pi to S3 and capture to write), not from EC2.

COOL evidence (EC2, 2026-10-22):

- AMI venv `/opt/cool/venvs/python_3.12`; `cv2` loaded from `/opt/cool/python_3.12/site-packages`
- `cv2.__version__` = 5.1.0-dev, `platform.machine()` = aarch64, 4 OpenCV threads on 4 vCPUs
- `getBuildInformation()` (saved as `cool_buildinfo.txt`): Custom HAL = carotene 0.0.1 + **KleidiCV 0.7.0**; **DNN MLAS** with ARM64 NEON SGEMM/SGEMV kernels; CPU baseline SVE, NEON, FP16, NEON_DOTPROD, NEON_FP16, NEON_BF16; pthreads
- AWS Marketplace product: Cloud Optimized OpenCV for AWS Graviton4, version 3.1 (from the instance's auto-created security group)

YOLOPv2 inference, same c8g.xlarge, 640 x 640 input (`dnn_bench.py`, median of 5 after warm-up):

Backend	Time per frame
OpenCV 5 + COOL, classic engine, FP16 target	**0.74 s**
PyTorch (TorchScript), earlier run	0.89 s
OpenCV 5 + COOL, classic engine, FP32	0.97 s
OpenCV 5 + COOL, new engine (FP32; it ignores targets)	1.20 s

OpenCV warns that the FP16 target "falls back" to FP32, yet it was 23 to 31% faster than classic FP32 at every instance size.

Scaling across Graviton4 sizes (classic engine, FP16 target, `src/scripts/dnn_bench.py`, 2026-10-22; full method in `phase2/ec2-compute-benchmark.md`):

Instance	vCPU	Inference	Speed-up	Parallel efficiency	On-demand $/h (approx.)
c8g.xlarge	4	0.742 s	1.00x	100%	0.16
c8g.2xlarge	8	0.418 s	1.78x	89%	0.32

Instance	vCPU	Inference	Speed-up	Parallel efficiency	On-demand $/h (approx.)
c8g.4xlarge	16	0.222 s	3.34x	84%	0.64

COOL's DNN path keeps using the added cores up to 16 vCPUs, so compute cost per frame rises only about 18% from the smallest size to the largest (about $0.033 to $0.039 per 1,000 frames). The FP16 target beat classic FP32 by 23 to 31% at every size (0.969, 0.565 and 0.320 s), despite OpenCV's fallback warning, so the gain is consistent rather than noise. Operating choice: c8g.4xlarge for drives and live demos, a smaller size the rest of the time, since live mode pays the hourly rate for the whole drive.

COOL vs standard OpenCV (c8g.4xlarge, 2026-10-22): no difference for inference. Same instance, same script, `opencv-python-headless==5.0.0.93` in a separate venv:

Setup	COOL 5.1.0-dev	Standard 5.0.0
Classic engine + FP16 target (used)	0.222 s	0.223 s
Classic engine, FP32	0.320 s	0.323 s
New engine	0.346 s	0.291 s

The standard aarch64 wheel already bundles KleidiCV (26.03, newer than COOL's 0.7.0) and the same DNN MLAS kernels, so on Graviton a plain `pip install` gets the Arm optimisations COOL advertises for this workload. The image operations around inference (undistort, resize, colour conversion, warp, sign candidates, JPEG) take about 7 ms per 640 frame (`src/scripts/ops_bench.py` , laptop), so COOL cannot change the frame time much there either. One reproducibility trap: the COOL AMI sets `PYTHONPATH` system-wide to its own `cv2` , which silently replaces any other OpenCV, even in a fresh venv; the baseline has to be run with that variable cleared.

Evaluation

The pipeline runs end to end from a moving car, but before the FP16 switch EC2 needed 1.3 s per frame against 1 frame/s arriving, so the delay grew.

Live run from the Pi feed, before the FP16 switch (session `20261022T140513Z` , c8g.xlarge, frames 1 to 20):

Stage	Seconds per frame
Download from S3	0.03 to 0.18
Inference (new engine)	1.21 to 1.23
Signs	0.00 (0.20 once, when candidates reached OCR)
Post-processing	0.04
Outputs	0.01 to 0.10
Total	**1.29 to 1.51**

- Capture-to-result delay was 4.1 to 5.4 s with an empty queue, and climbed to 7.4 s once 2 frames were waiting. About 1.3 s is processing; the other 3 to 4 s is cellular upload, the S3 event, Lambda and SQS, and is not yet broken down.
- Frame #0 showed 66,973 s of lag: a message left in the queue from the previous day. Exclude each session's first frame from latency figures.
- Every frame in that run reported no lanes. Still to confirm whether the car was on marked road.

Latency fixes, 2026-10-22: capture-to-result delay went from about 3.2 s to about 0.84 s with the virtual Pi on a laptop, and about 1.2 to 1.4 s from the real Pi once its own capture time was included (below).

1. **Lambda wasn't the slow part.** CloudWatch over 806 calls: 107 ms average (p95 170 ms), 5 cold starts averaging 486 ms. The 1 to 1.2 s `s3_to_queue` hop was S3 delivering the event to Lambda, which AWS gives no setting for.
2. **Direct enqueue:** the uploader PUTs the frame to its archive key and sends the SQS message itself, in the format the Lambda used, so S3 events and Lambda leave the live path. Tested with the virtual Pi (`src/scripts/virtual_pi.py --direct`), which replays a recorded drive in real time and uploads like the Pi.
3. **Bigger instance** for inference (see AWS deployment and COOL).

Hop (one frame, virtual Pi from the laptop, `--direct`)	Seconds
Capture to S3 (wait for upload + PUT)	0.38
S3 to queue	0.00

Hop (one frame, virtual Pi from the laptop, `--direct`)	Seconds
Waiting in queue	0.10
Processing on EC2	0.36 (download 0.04, inference 0.26, signs 0.01, post-processing 0.04, outputs 0.01)

The upload from the laptop's connection is now the largest hop, and it will be larger over the Pi's cellular link. The 0.26 s inference is consistent with the c8g.4xlarge benchmark (0.22 s).

Capture size: record at 1280, upload at 640 (decided 2026-10-22). The Pi 3B records 1280 x 1280 JPEG q90 (about 190 KB per frame) and the uploader downscales each live frame to 640 x 640 q70 (about 25 KB) with the JPEG decoder's built-in scaling, in 73 ms. Capturing at 640 directly would cut the Pi's encode load and save about 0.1 to 0.2 s of delay, but it would turn the CAN-synced recordings into 640 too and change a capture setup proven at 0.64% dropped frames. It is the fallback if the highway drive shows throttling or drops; it would need the sensor mode fixed to 1640 x 1232, so the 640 image covers exactly the calibrated field of view.

Capture time is now part of the lag. The recorder stamps the preview file with the sensor's capture time, and the uploader sends both that and the time the file was written. The live page shows the capture-to-write part (JPEG encode plus any SD-card backlog) as its own row, so every lag figure starts at the sensor. Measured on the Pi 3B: 0.22 to 0.56 s, JPEG encoding of the 1280 recording frame plus the recorder's disk queue (not yet split).

Measured with the real Pi, direct mode, 640 uploads (2026-10-22): once capture time was included, a frame took about **1.2 to 1.4 s from camera to result**. The ~0.9 s above came from the laptop's virtual Pi and left out the Pi's own encode and write time. Uploader log, three consecutive frames:

Step on the Pi	Frame 117	Frame 118	Frame 119
Capture to write (encode, disk queue)	0.56 s	0.24 s	0.22 s
Resize to 640	0.09 s	0.10 s	0.12 s
S3 PUT	0.30 s	0.57 s	0.37 s
SQS send	0.11 s	0.38 s	0.24 s
Age when queued	1.08 s	1.32 s	1.03 s

Two further cuts are designed but not built: carrying the ~18 KB frame inside the SQS message (one network call instead of two, about 0.3 to 0.6 s saved), and writing the 640 preview straight from the camera's pixels (about 0.3 s saved). Together they would bring the end-to-end lag to roughly 0.6 to 0.9 s.

Sign reader (`src/scripts/eval_signs.py`, 640 x 640 frames like the live pipeline, 2026-10-22; results in `data/eval/`):

- **Real stop signs:** found in 26 of 83 frames of the September stop-sign approach, and at a second intersection (University Ave) in the October drive.
- **False alarms: 0 in 8,001 frames** sampled from every local drive. The sweep flagged 4 frames, and all 4 are real stop signs at that second intersection, checked by eye.
- **Speed limits, synthetic** (none of my roads had a speed-limit sign in view): the public-domain MUTCD R2-1 sign from the FHWA, with each legal value drawn in, pasted into real frames with blur, rotation, the camera's colour cast and JPEG compression; 1,964 signs over 5 seeds.

Sign width	Read correctly	Misread
30 to 40 px	0% (skipped by design)	0
40 to 50 px	33%	0
50 to 70 px	57%	0
70 to 85 px	93%	1
85 to 100 px	99%	1
100 to 140 px	99%	1

Three misreads in 1,964 (0.15%), for example 75 read as 15. Cost: about 16 ms per frame on a laptop when nothing sign-like is in view.

Lane detection against hand labels (session `20261022T164127Z-084b`, `src/scripts/label_lanes.py`, 2026-10-22): 150 frames sampled evenly from moments the car was moving faster than 5 mph, labelled without seeing the pipeline's output; 55 marked "can't tell" and left out, 95 scored. Frames were downscaled to 640 x 640 like the Pi's uploads.

Measure	Result
`lane_valid` precision (says "both lines found" and both were visible)	**0.977** (43 of 44)
`lane_valid` recall (both lines visible and found)	**0.729** (43 of 59)
`lane_valid` accuracy	0.821 (78 of 95)
Departure agrees, where both see the lane	41 of 43

The pipeline is conservative: when it reports a lane it is almost always right, but it misses about a quarter of frames where a person sees both lines. Of the 2 departure disagreements, 1 was a car labelled as drifting left that the pipeline called departing right, and 1 a centred car called departing right. The labelled set is mostly centred driving (57 centred, 2 drifting left, none right), so departure accuracy is barely tested yet. Every disagreement is saved as an overlay plus bird's-eye image in `lane_eval/failures/` for the failure-case section.

Still to measure: the same lane evaluation on the highway drive (and more departure examples), p50/p95 delay after the FP16 switch, real speed-limit signs from the highway drive, and cost per drive-hour.

Demo and user experience

Judges run the car themselves: `src/scripts/virtual_pi.py` replays a recorded drive on the judge's own computer and uploads frames exactly as the Pi does, through the real AWS pipeline, so the optimised path and its lag are visible end to end.

- **Screen:** the undistorted camera frame next to a CAN dashboard from the same instant: speed (mph and km/h), gear (P R N D L), RPM, a steering wheel that turns with the real angle, accelerator and brake bars, and flashing turn-signal arrows. A clickable

timeline and arrow keys seek through the drive; space pauses (uploads pause too).

- **Verified against the camera:** the steering wheel's direction was checked against image motion (positive angle = scene slides right = turning left), and the turn-signal codes against the steering that followed them (1 = left, 2 = right).
- **Side by side:** the frame overlay carries a clock and the last upload's timings, so placing it next to the live results page shows the end-to-end lag directly.
- **Results page (live):** www.initiativelabs.cc/ivision-demo, password protected. It starts the EC2 instance on request (about 1 to 2 minutes), shows the live view over HTTPS from `demo.initiativelabs.cc` (Elastic IP, Caddy, signed expiring token), and offers the session's videos for download 30 s after the drive ends. The instance stops itself after 20 idle minutes.

How a judge runs it (README, "For judges"): download the privacy-processed drive (503 MB, release v1.0), open the results page and press Start, then run `python src/scripts/virtual_pi.py 20261022T164127Z-084b --demo`. The virtual Pi asks for the demo password and receives one-hour AWS credentials from the website (STS, assumed through Vercel's OIDC federation) that can only upload frames and send to the frame queue, the same rights the real Pi has. No AWS account and no car needed.

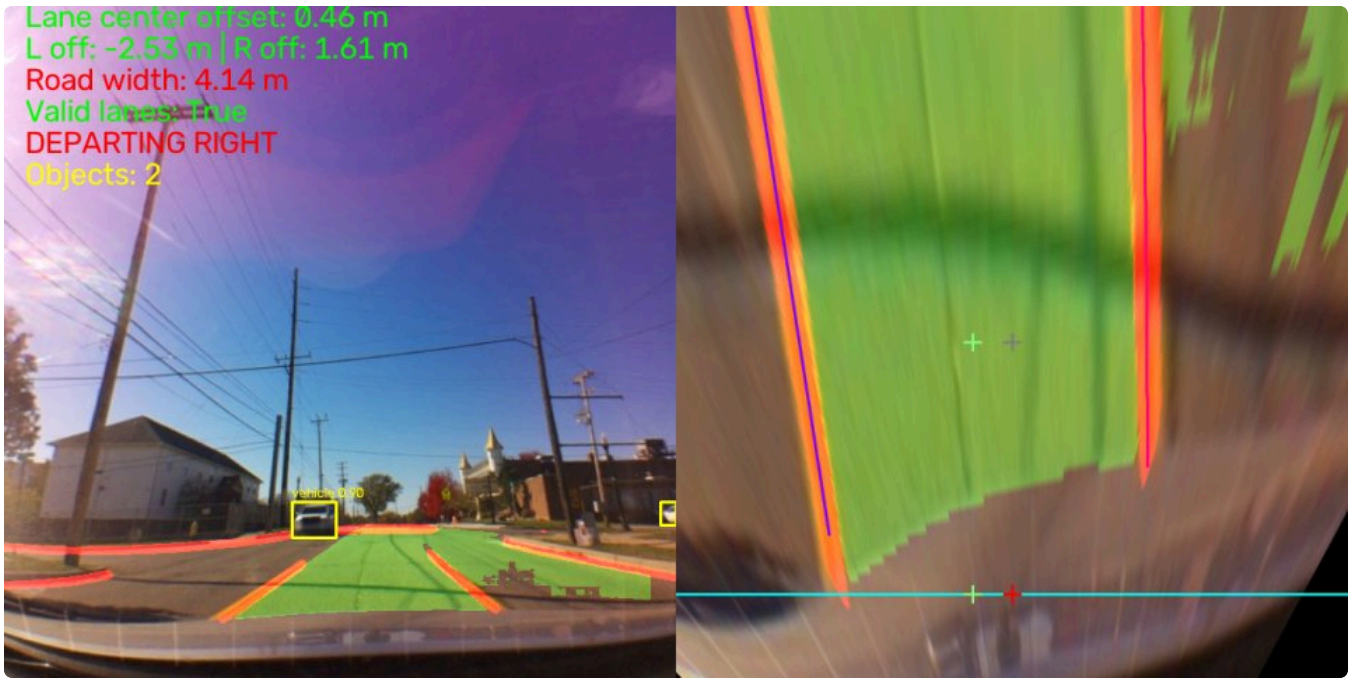
Limitations and failure cases

The biggest limit is delay: about 1.2 to 1.4 s from camera to result, most of it on the Pi and its network link (encode, upload, queue send), so this is an informative system, not a warning system.

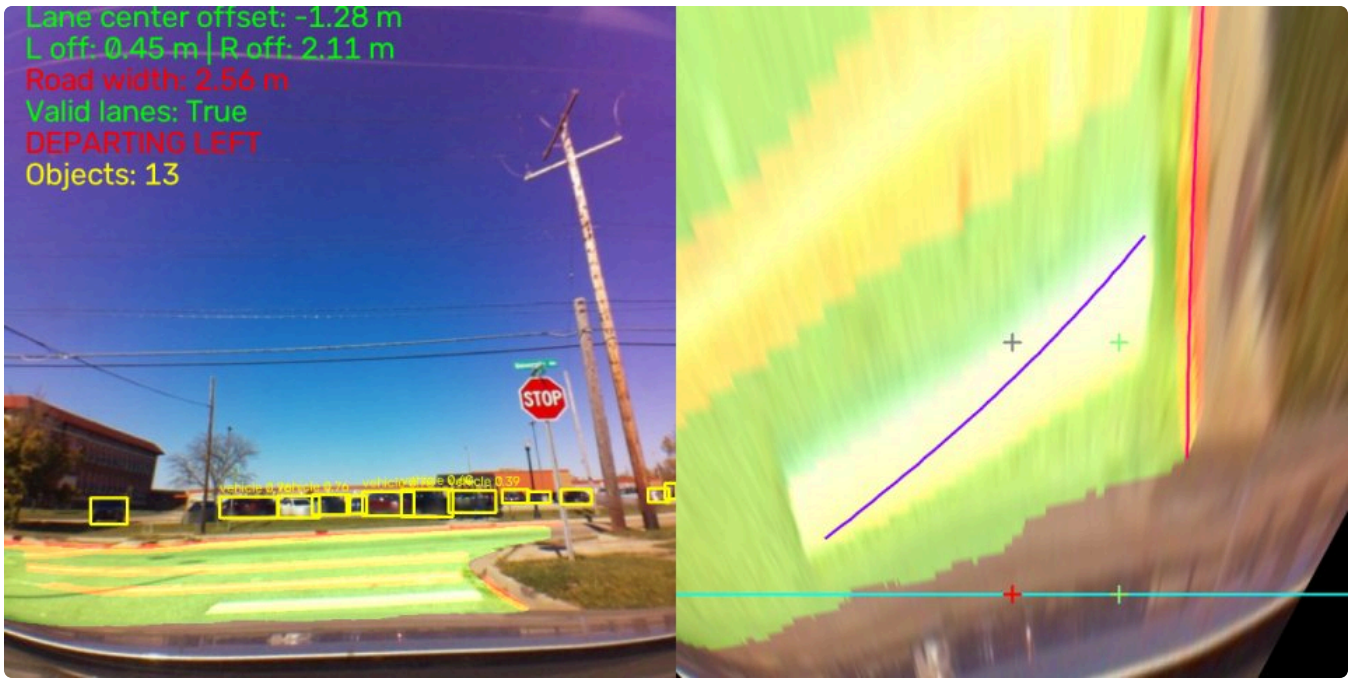
- **Delay:** fine for logging, review and fleet monitoring, but too late for an in-car warning. The queue part is fixed by being faster than 1 frame/s or by skipping to the newest frame; the transport part needs measuring.
- **Coverage:** in cellular dead zones frames never reach S3, so there are gaps in the results.
- **Detector classes:** YOLOPv2 finds vehicles only. It was trained on BDD100K with every traffic object merged into one class, so on 81 frames it boxed no people, traffic lights or signs, even where they were visible.
- **Signs:** speed limits are validated only on synthetic signs so far. Not handled: yellow advisory signs, "MINIMUM" signs, night glare, signs seen at an angle. Signs under 48 px are skipped on purpose (at 40 px "55" was read as "65"). At 1 frame/s a sign may be readable in only 1 or 2 frames.

- **Calibration:** the bird's-eye view is valid only for the camera pose used in the cone calibration, and only calibrated to about 7 m ahead.
- **Known anomalies to investigate:** a frame marked lane-valid with the left line 3.99 m away and "departing right" (probably the next lane's line), and a "departing left" at an intersection.
- **Data from before the missing-lane fix** (2026-10-21) still has -4 m offsets in its `metrics.json`.

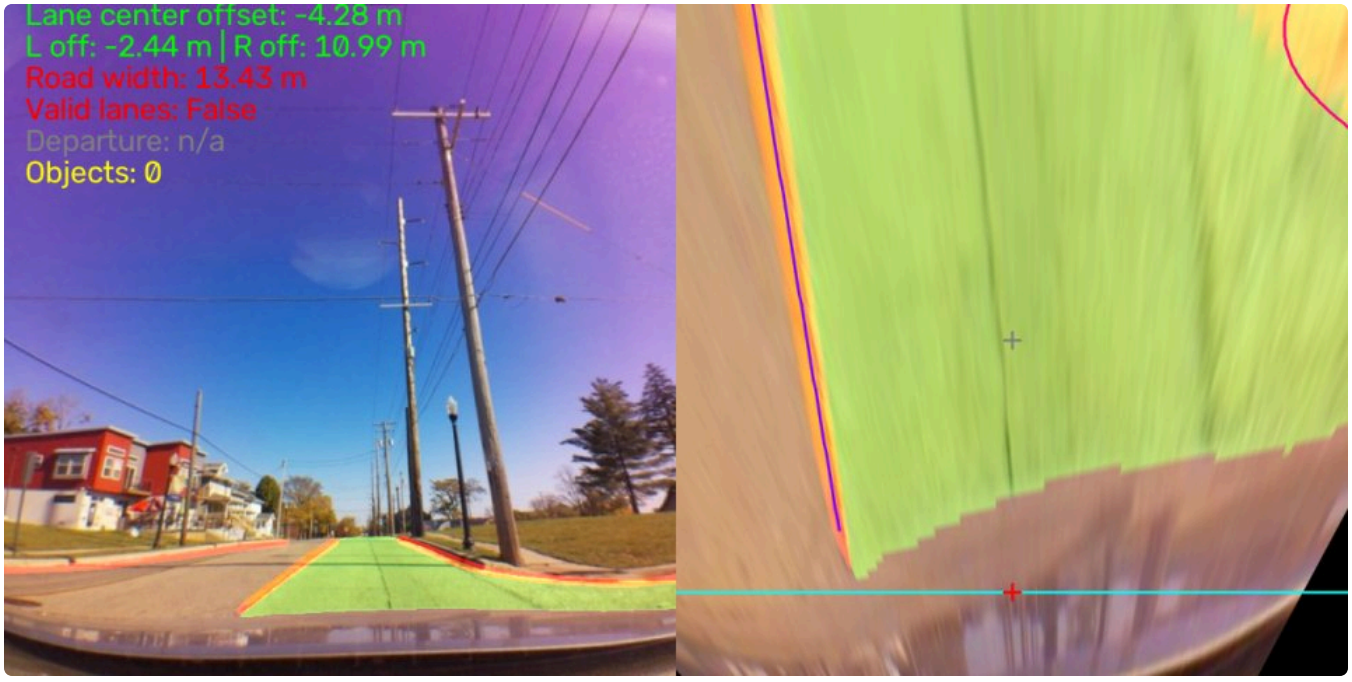
Failure cases (from the hand-labelled evaluation; shown on the privacy-processed frames, camera overlay left, bird's-eye view right; all images in `docs/failure-cases/`):



Wrong departure. A narrow street with no centre line: the curb is taken as the right lane line, so the lane looks 4.1 m wide and the car 0.46 m right of centre, just past the 0.4 m threshold. It reports "departing right" while the car is centred.



False lane. At a stop sign, the crosswalk stripe is read as a lane line and the pipeline reports "departing left". The sign reader flags the stop sign correctly in the same frame; combining the two (no departure warning at a stop line) is an easy fix.



Missed lane. The right line is picked up on the far side of a wide road, 11 m away. The 13.4 m road width fails the sanity check, so no lane is reported: cautious rather than wrong, and the most common failure (16 of 19).

Responsible use

Initiative Vision is an observation and data system, not a safety system: it never controls the vehicle, and the driver must not rely on it.

- **No actuation:** results go only to a web page and to S3
- **Shared data is anonymised** (`src/scripts/privacy_package.py`): faces blurred with YuNet from the OpenCV model zoo; number plates covered by blurring the lower half of every vehicle box from YOLOPv2 that is large enough for a readable plate (the zoo's plate detector, trained on Chinese plates, found none on this footage); GPS latitude and longitude removed from the decoded CAN; raw CAN left out entirely, because it carries the vehicle's VIN and the OnStar position. On a 50-frame check, YOLOPv2 still found 93% of vehicles in the blurred frames and lane results were unchanged, so the shared data still works for the demo. Before/after samples are reviewed by hand before publishing.
- **Access:** least-privilege IAM per component (the Pi may only put frames under `live/` and send to one queue; the instance role may only read frames, write results, consume the queue and publish metrics into one namespace). Raw frames stay in a private bucket. To do: replace the root access keys used on the development laptop with an IAM user.
- **Retention:** raw frames under `live/` to be expired by an S3 lifecycle rule (30 days); results under `lld/` are already processed.
- **Driver distraction:** the live page is for a passenger or a remote viewer, not the driver
- **Cost control:** the demo instance stops itself after 20 idle minutes (no frames, no viewers); between drives it is stopped

Future work

- **The research goals the Vision 1 was built for:** use the recorded CAN and GPS as ground truth to tune the plant model and controllers of a simulated vehicle in MATLAB, and extract scenarios (lane changes, stops, curves) from the drives to validate controllers quickly.
- **Alerts back in the car:** a small OLED screen on the Pi showing lane position, departures and the last sign, fed by a second SQS queue of results (designed, not built).

- **Under a second end to end:** carry the ~18 KB frame inside the SQS message (one network call instead of two) and write the 640 preview straight from the camera's pixels; expected 0.6 to 0.9 s.
- **Fewer lane failures:** suppress departures at stop lines and crosswalks (the sign reader already sees the stop sign), and treat curbs differently from painted lines.
- **More of the road:** a second detector for pedestrians, cyclists and traffic lights; distance to the vehicle ahead from the bird's-eye view; using live CAN to flag a departure without a turn signal.
- **More evaluation:** highway driving with lane changes, real speed-limit signs, night and rain.
- **Operations:** an S3 lifecycle rule on raw frames, an alarm on the dead-letter queue, and the AWS resources as one infrastructure template.